End-to-End Cloud Solution Design & Deployment

Azubi Africa, Portfolio Project 1 (GitHub & Trello) | Project plan: steps, phases and roles

1. The Customer Problem

A fast-growing startup has launched a new web application, but users are experiencing:

- Slow page load times
- Inconsistent availability during peak traffic
- Concerns about the security of the application
- No structured deployment process, which makes collaboration difficult and increases the risk of deployment errors

They need a **secure, highly available and scalable** cloud solution that can be deployed within a **limited budget** and support future growth.

2. Project Objective

Design, architect and deploy a secure, scalable web application on AWS using **CloudFront, ALB and related services**. We will collaborate using **GitHub** for version control and **Trello** for project management, building real-world cloud deployment skills within the **AWS Free Tier**.

3. The Web Application: UPSA Hostel Visitor Management System

The app we will deploy is a Next.js system for UPSA hostels and campus. It handles visitor check-in and check-out, package drop-off with resident alerts, and lecturer appointment booking, with Supabase (PostgreSQL + Auth) as the backend.

What it does

Feature	Who uses it	What happens
Visitor check-in (public)	Visitors, porters	Visitors check in with their name, phone, the resident they are visiting, room and hostel. Porters check them out when they leave.

Feature	Who uses it	What happens
Packages	Porters, residents	Porters register packages and residents are notified (hook for SMS or email). Residents mark packages as collected.
Lecturer appointments	Students, lecturers	Students book meetings based on lecturer availability. Lecturers and students see appointments on the dashboard.

Tech stack

- **Frontend:** Next.js 14 (App Router), TypeScript, Tailwind CSS
- **Backend and database:** Supabase (PostgreSQL, Auth, Row Level Security)
- **User roles:** Admin, Porter, Resident, Lecturer, Student

Pages

Route	Purpose
`/`	Home
`/check-in`	Public visitor check-in
`/login`	Login for staff, residents, lecturers and students
`/dashboard`	Role-based view of visits, packages and appointments
`/dashboard/packages`	Porter registers a package
`/appointments`	Student books a lecturer

Why we chose this app and stack

These reasons are drafted from the app's design. Edit them so they match your team's own thinking.

- **It is a real, simple campus problem.** Visitor logging, package handling and lecturer booking are easy to explain in the Phase 5 presentation, so the focus stays on the cloud deployment.
- **One codebase for pages and server logic.** Next.js serves the pages and the API routes, such as the package alert route, so there is only one app to deploy to EC2.

- **No database server to run.** Supabase provides managed PostgreSQL and login, which saves setup time and keeps us inside the Free Tier budget.
- **Security built in.** Row Level Security and five user roles let each person see only what they should, which matches our least-privilege approach on AWS.
- **It shows the architecture working.** The public check-in page gets busy at peak times, so it gives us a natural way to demonstrate CloudFront caching, the load balancer and Auto Scaling.

Configuration the app needs

Variable	What it is	How we handle it
`NEXT_PUBLIC_SUPABASE_URL`	Project URL from Supabase, Settings, API	Safe in the browser
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	The anon public key	Safe in the browser, protected by Row Level Security
`SUPABASE_SERVICE_ROLE_KEY`	The service_role key, used only for seeding	Secret. Never commit it to GitHub or put it on the live server

Setup in short: create a Supabase project, run `supabase/schema.sql` in the SQL Editor, copy `.env.example` to `.env` and fill in the keys, then run `npm install` and `npm run dev`. Optionally run `npm run db:seed` to create demo users for admin, porter, resident, lecturer and student.

How it fits our AWS design

- Next.js runs on the EC2 instances with nginx in front, and the ALB sends traffic to it. We need to confirm this setup, since our design slides describe a simple nginx app.
- Supabase is hosted outside AWS, so the app reaches it over HTTPS. We should add it to the architecture diagram as an external service.
- Store the environment values in GitHub Secrets or AWS Systems Manager Parameter Store, never in the repository.
- The demo seed accounts all share one simple password. Do not use them on the live HTTPS site: change the passwords or remove those accounts first.
- Next.js serves its own static files from `/_next/static/`, so CloudFront should cache that path as well as the S3 `/static/*` path.

- The Send alert action only records the time until an SMS or email provider is added. For the demo it can stay as a documented future step.

4. Beginner Guide: How GitHub, AWS and Supabase Work Together

The web app is already built, so our job is to **deploy** it: get it running on AWS, secure it, and make updates automatic. Think of the app as a recipe and AWS as the restaurant.

What each tool does

Tool	Its job	Does the app run here?
GitHub	Stores the code, keeps every change, and runs the GitHub Actions automation	No
AWS	Runs the app (EC2), shares traffic (ALB), speeds up and secures access (CloudFront), stores static files (S3)	Yes
Supabase	Keeps the database and the logins	No, it is a separate hosted service
Trello	Tracks who is doing which task	No

Pushing versus deploying

- **Pushing** means sending code to GitHub so it is stored and shared. Nobody can visit the app there.
- **Deploying** means taking the code from GitHub and getting it running on the AWS servers so visitors can use it.

How GitHub is linked to AWS

- **GitHub Actions** is an automation robot inside GitHub. We give it a to-do list in the file `.github/workflows/deploy.yml`, and it runs whenever code is merged into `main`.
- **Permission** comes from OIDC. In AWS we create an IAM role that trusts only our GitHub repository. When the robot runs, AWS gives it a temporary pass that expires, so no permanent AWS keys are stored anywhere.
- **The command to the servers** is sent through SSM Run Command, which lets AWS give a server instructions without anyone logging in to it.

What happens when we change the code

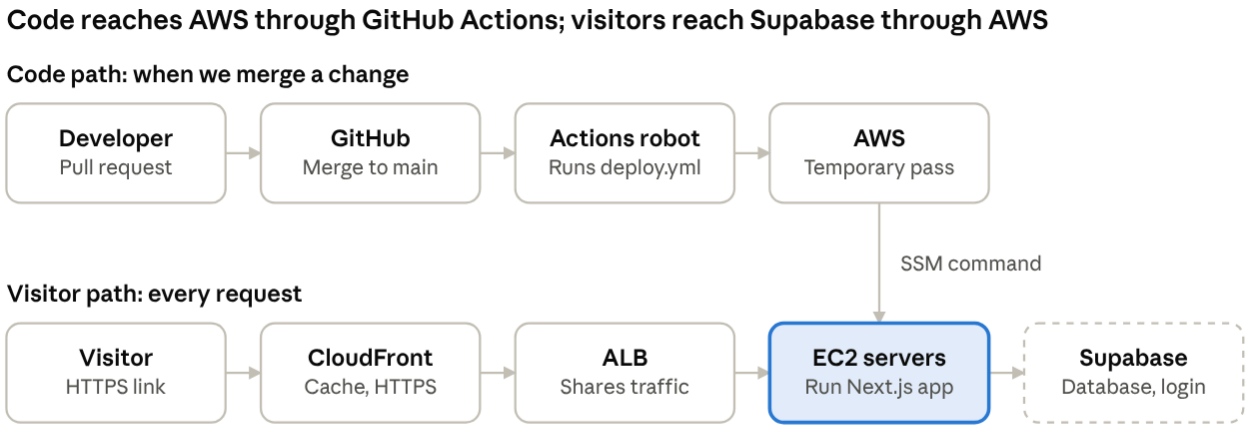
1. A developer works on a feature branch and opens a pull request.
2. A teammate reviews it and merges it into `main`.
3. The merge starts GitHub Actions.
4. The robot asks AWS for a temporary pass using OIDC.
5. It sends a command to the EC2 servers through SSM.
6. Each server downloads the latest code, installs it, builds it and restarts the app.
7. The robot checks the health endpoint, and the live site shows the change.

Two details to plan for: if the repository is private, the servers need a way to read it (for example a deploy key). Also, when Auto Scaling starts a brand new server, a startup script in the launch template must fetch and start the app by itself.

Order of work

1. **Create the Supabase project** and run `supabase/schema.sql`. (Web Developer)
2. **Put the code on GitHub** with the branching strategy and the README. (DevOps Engineer, Documentation Lead)
3. **Set up Trello and AWS**: IAM users, budget alert, certificate request. (Project Manager, Cloud Architect)
4. **Run the app on one EC2 server first**, with the Supabase values in its environment settings and nginx in front. Prove it works before adding more. (Web Developer, Cloud Architect)
5. **Add the ALB and Auto Scaling** so two servers run in two zones. (Cloud Architect)
6. **Add CloudFront and HTTPS**, lock down the security groups, and set the CloudFront web address as the allowed site URL in Supabase so logins work. (Cloud Architect, Web Developer)
7. **Automate deployment** with GitHub Actions and test it with a small change. (DevOps Engineer)
8. **Add alarms and collect evidence** for each test. (DevOps Engineer, Documentation Lead)
9. **Present, then clean up** the AWS resources. (All, DevOps Engineer)

How it all fits together



Highlighted: EC2 servers, where both paths meet. Dashed: Supabase, hosted outside AWS.

code path and visitor path · 9 boxes

Read the top row left to right for how a change reaches AWS, and the bottom row for how a visitor reaches the data. The two meet at the EC2 servers.

Why Supabase stays outside AWS

The app was built around Supabase, so its logins, database queries and security rules (Row Level Security) all expect it. We keep it as a managed database service. The EC2 servers reach it over the internet using the keys in their environment settings, and that is the only link needed. Moving the data to an AWS database such as RDS would mean rewriting the login and data code and adding cost, so it is not part of this project.

Public or private subnet?

- A **public subnet** has a direct route to the internet. A **private subnet** does not, so it is harder to attack from outside.
- A **NAT Gateway** lets servers in a private subnet go out to the internet (for example, to reach Supabase) while blocking anything from coming in.
- Our app must reach Supabase, so servers in private subnets would need a NAT Gateway. It is not covered by the Free Tier and costs roughly $30 or more per month, so check the AWS pricing page for the current figure.
- **Our choice:** public subnets in two zones, locked down with security groups so the EC2 servers accept traffic only from the load balancer. Private subnets with a NAT Gateway are a documented later upgrade.
- If the mentor wants the private setup, build it for the demo and delete the NAT Gateway the same day.

5. Our Solution at a Glance

Every component exists to answer one specific problem, and each has a test that proves it works.

Customer problem	Our design response	How we will prove it
Slow page loads	CloudFront edge caching, with static files in S3	Before/after page-load tests; cache-hit headers
Unavailable at peak traffic	Application Load Balancer (ALB) + Auto Scaling Group (min 1 / desired 2 / max 3) across 2 Availability Zones	Stop one instance and traffic continues on the other
Security concerns	HTTPS (ACM), CloudFront-only access, scoped security groups, private S3, IAM least privilege	curl tests; direct ALB and S3 access is blocked
No deployment process	GitHub branching + pull requests + GitHub Actions	A push to main updates the live site
Limited budget, future growth	Free Tier-sized instances, ASG cap, AWS Budgets alerts, modular design	Budget alert active; clean-up owner named

How a request travels

1. **Browser to CloudFront:** HTTPS only. HTTP is redirected. Cached content is returned here.
2. **CloudFront to ALB:** uncached/dynamic requests, over HTTPS only.
3. **CloudFront to S3:** `/static/*` files come from a private bucket via Origin Access Control (OAC), as a parallel branch.
4. **ALB to EC2:** a healthy instance in either Availability Zone serves the request on port 80 inside the VPC.

Result: faster delivery, balanced load, and resilience if an instance or a whole zone fails.

Why each component is needed

Component	Role	If we left it out
CloudFront	Edge cache, single secure entry point, TLS	Slower site and exposed origin

Component	Role	If we left it out
ALB	Balances traffic, health checks, TLS	Single point of failure
EC2 + Auto Scaling	Runs the Next.js app behind nginx, scales and self-heals	Nowhere to run the app

Options we considered and rejected: a single EC2 (no resilience), ALB only (no CDN, exposed), and CloudFront straight to EC2 (no load balancing).

6. Project Phases

The project is delivered in five phases. Phase 1 and Phase 5 are taken directly from the course page. Phases 2 to 4 are grouped from the architecture proposal, so please match their names to the course page if they differ.

Phase 1: Project Setup & Environment Configuration

Challenge: set up all tools, accounts and repositories for the project.

Activities

- Create and configure the AWS Free Tier account, and set up IAM users with least-privilege permissions
- Install and configure the AWS CLI locally with named profiles
- Create a GitHub repository with a proper branching strategy (main, develop, feature branches)
- Set up a Trello board with columns: Backlog, In Progress, Review, Done, and add all team members
- Launch an EC2 instance (t2.micro) and configure security groups to allow HTTP/HTTPS traffic
- Create an S3 bucket for static assets with appropriate public access settings
- Document the project environment setup in a README.md file in GitHub
- Request a free SSL/TLS certificate via AWS Certificate Manager (ACM) for a custom domain

Phase 2: Build the Application Layer (proposed)

- Deploy the Next.js hostel visitor app on EC2 (with nginx in front) and confirm it responds
- Create the Application Load Balancer and a target group with health checks

- Create the Auto Scaling Group (min 1 / desired 2 / max 3, CPU target 50%) across 2 Availability Zones
- Upload static files to S3 under `/static/`
- Test: stop one instance and confirm the site stays up

Phase 3: Delivery and Security (proposed)

- Create the CloudFront distribution with the ALB and S3 as origins
- Attach the ACM certificates (us-east-1 for CloudFront, regional for the ALB) and redirect HTTP to HTTPS
- Lock down access: the ALB accepts only CloudFront, EC2 accepts only the ALB, S3 stays private via OAC
- Turn on MFA for the root account, use an EC2 instance role (no keys on servers), and enable S3 Block Public Access
- Test: curl redirect test, and confirm direct ALB and S3 access is blocked

Phase 4: CI/CD, Monitoring and Cost Control (proposed)

- Build the GitHub Actions workflow: a pull request into protected main, then a push to main triggers the deploy
- Deploy via SSM Run Command with GitHub OIDC (no open SSH, no stored keys), sync `/static` to S3, and check the health endpoint
- Set up CloudWatch alarms (EC2 CPU above 80%, ALB 5xx rate above 5%), with email alerts through SNS
- Set an AWS Budget with alerts at 80% actual and 100% forecast
- Agree the clean-up (teardown) order and owner

Phase 5: Intern Presentation, Project Showcase

Challenge: present your completed cloud project, explain your architectural decisions, and demo the live deployment.

What to present

- Problem statement: what problem the web app solves
- Architecture walkthrough: CloudFront to ALB to EC2 flow
- Security decisions: how the deployment is secured end to end
- CI/CD demo: a live GitHub Actions pipeline run
- Trello board review: completed sprints and task history
- Live demo: access the deployed app through the CloudFront URL (HTTPS)
- Challenges faced: blockers encountered and how they were resolved

- Lessons learned and what we would do differently next time

7. Team Roles

The team has five roles. Everyone helps test their own work, and everyone should be ready to explain their part in the Phase 5 presentation.

Role	Responsibilities	Main phases
Project Manager	Runs the Trello board and sprints, assigns cards, tracks progress, records decisions and owners, and keeps the team on schedule	All
Cloud Architect	Owns the architecture diagram and design decisions, builds the AWS infrastructure (EC2, ALB, Auto Scaling, S3, CloudFront), and owns security: IAM, security groups, ACM certificates and OAC	1, 2, 3
Web Developer	Builds and maintains the Next.js hostel visitor app and its Supabase database, adds a health check endpoint, and fixes bugs found in testing	2, 3, 4
DevOps Engineer	Sets up the GitHub repo and branching, builds the GitHub Actions pipeline, handles deployments and rollback, and sets up CloudWatch alarms, SNS alerts and AWS Budgets. Owns clean-up after the demo	1, 4
Documentation Lead	Writes the README and setup guide, collects evidence and screenshots, keeps the architecture docs, and prepares the slides and demo script	1, 5

8. Project Management Model: Agile on a Kanban Board

We are using **Agile**: short sprints, one per project phase, tracked on a **Kanban board** in Trello. This matches the course setup, where the Trello board has the columns Backlog, In Progress, Review and Done, and the final presentation includes a review of completed sprints.

Why Agile with a Kanban board

- The work comes in phases that build on each other, so we deliver and test one piece at a time instead of one big handover at the end.

- The board shows everyone what is being done, who owns it and what is blocked.
- Every card passes through Review, which fits the pull request process in GitHub.
- The Trello history becomes the evidence for the Phase 5 board review.

How we run it

1. **Sprint = one phase.** Each phase gets a goal and a set of cards in the Backlog.
2. **Plan.** The Project Manager and the team pick the cards for the sprint and give each one an owner.
3. **Work.** Cards move from Backlog to In Progress to Review to Done.
4. **Review in GitHub.** Changes go through feature branches and pull requests into main, and a teammate reviews before merging. Nobody pushes straight to main.
5. **Sprint review.** We run the phase's test, attach the evidence (screenshot or curl result) to the card, and record decisions in Trello and the README.
6. **Retrospective.** We note what went well and what blocked us, which feeds the challenges and lessons learned in the final presentation.

Board rules

- A card is Done only when it is tested and has its evidence attached.
- Keep work in progress small: each person works on one or two cards at a time.
- Stay on Free Tier-sized resources, watch the budget alerts, and clean up in the agreed order after the demo.

9. Deliverables

- GitHub repository with all source code
- Completed Trello board (screenshot)
- Architecture diagram
- Live HTTPS URL served through CloudFront
- README with a setup guide
- Presentation slides
- Recorded demo video (optional)

10. Risks and How We Handle Them

Risk	How we handle it
Surprise AWS charges	Budget alerts, ASG cap of 3 instances, named clean-up owner
Certificate region or validation delays	Request certificates early in Phase 1
CloudFront 502/504 errors	Make sure the origin certificate matches the origin name, and test before locking down access
HTTP between ALB and EC2	Accepted risk, since traffic stays inside the VPC and EC2 accepts only the ALB

11. Decisions to Agree Before We Start

Decision	Recommendation
Region and domain	One region for everything, with the CloudFront certificate in us-east-1. A custom domain is needed for ACM certificates.
Network model	Public subnets in 2 AZs, locked down with security groups (no NAT cost). Private subnets as a later upgrade.
Deployment method	SSM Run Command via GitHub OIDC, with restricted SSH only if the mentor requires it
Auto Scaling	Min 1 / desired 2 / max 3, CPU target 50%
Cost guardrail	Agree a monthly budget and confirm the ALB is covered by our Free Tier model

12. Next Steps

1. Approve the design and assign owners
2. Phase 1: setup and certificates
3. Build and validate phase by phase
4. Demo with evidence